

Asymptotic Notations

Analysis of Algorithms

Complexity

Complexity is about *resource consumption* and there are two resources we care about:

Time complexity: how many elementary operations does the algorithm perform? We model this as a count of basic steps: *comparisons, assignments, arithmetic operations*. We treat each elementary operation as taking one unit of time. This is an *abstraction* real operations take different amounts of time on real hardware but it is a clean and useful model.

Space complexity: how much memory does the algorithm use beyond its input? *Every variable, every array, every recursive call stack frame costs memory*. Space complexity measures that cost as a function of n .

In both cases the question is the same: *as n grows, how does the cost grow?* Does it stay flat? Does it double every time n doubles? Does it square? Does it explode exponentially? The answer to that question is the *complexity* of the algorithm.

What Does Asymptotic Mean?

The word *asymptotic* comes from mathematics. An *asymptote* is a line or curve that another curve approaches but never quite reaches as you go to infinity. When we say asymptotic analysis, we mean: *we are studying the behavior of functions as n approaches infinity.*

Why infinity? Because we want to capture the long-run behavior what happens when inputs get large. For small inputs, almost any algorithm is fast enough. The interesting, practically relevant question is: what happens when n is ten thousand, a **million**, a **billion**? That is where algorithmic choices become life-or-death for a system.

Asymptotic Notations

Asymptotic notations are a mathematical language for describing the growth rates of functions. They give us a standardized, precise vocabulary for making statements like:

"This algorithm's running time grows no faster than n squared" Big- O

"This algorithm's running time grows no slower than $n \log n$ " Big- Ω

"This algorithm's running time grows at exactly the same rate as $n \log n$, up to constant factors" Big- Θ

Asymptotic Notations

Asymptotic analysis focuses on the rate of growth of an *algorithm's* runtime or space requirements.

Instead of measuring exact seconds (which vary by hardware), we use functions to describe how the number of operations increases with input size

The core philosophy of asymptotic analysis:

drop lower-order terms, drop constant factors, keep the dominant growth pattern.

Asymptotic Notations

$$T(n) = 4n^2 + 7n + 23$$

operations on an input of size n . Now let's ask: for large n , which term matters?

$$\text{At } n = 1,000$$

$$4n^2 = 4,000,000$$

$$7n = 7,000$$

$$23 = 23$$

The $7n$ term is 0.17% of the total. The constant 23 is essentially zero. The n^2 term is everything.

At $n = 1,000,000$, the $7n$ and 23 terms are even more negligible. As n grows toward infinity, the n^2 term completely dominates the other terms become rounding errors.

Asymptotic Notations Properties

- Categorize *algorithms* based on asymptotic growth rate e.g. linear, quadratic, polynomial, exponential.
- Ignore small constant and small inputs.
- Estimate upper bound and lower bound on growth rate of time *complexity* function.
- Describe running time of *algorithm* as n grows to.
- Describes behavior of function within the limit.

Limitations:

- Not always useful for analysis on fixed-size inputs.
- All results are for sufficiently large inputs.

Growth Rate of Functions

The **Growth Rate** (or Order of Growth) defines how the running time of an *algorithm* increases as the input size n becomes very large.

It is the most critical factor in comparing *algorithm* efficiency because hardware-specific constants become irrelevant at scale.

We evaluate performance as $n \rightarrow \infty$.

An *algorithm* with a slower growth rate is considered superior for large datasets.

Big-O Notation

The upper bound

We say $f(n) = O(g(n))$ if there exist positive constants **c** and **n_0** such that:

$$0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0$$

$f(n) = O(g(n))$ means function $g(n)$ is an asymptotically upper bound for $f(n)$.

We may write $f(n) = O(g(n))$ OR $f(n) \in O(g(n))$

Set of all functions whose rate of growth is the same as or lower than that of $g(n)$.

Big-Omega Notation (Ω)

The Lower Bound

$f(n) = \Omega(g(n))$ if there exist positive constants c and n_0 such that:

$$0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0$$

$f(n) = \Omega(g(n))$ means function $g(n)$ is an asymptotically lower bound for $f(n)$.

We may write $f(n) = \Omega(g(n))$ OR $f(n) \in \Omega(g(n))$

Set of all functions whose rate of growth is the same as or higher than that of $g(n)$.

Big-Theta Notation(Θ)

The Tight Bound

Big- Θ is where both Big-O and Big- Ω come together. It provides a *tight* bound a precise characterization of *growth* rate.

$f(n) = \Theta(g(n))$ if and only if:

$$f(n) = O(g(n)) \text{ AND } f(n) = \Omega(g(n))$$

Expanding this out: there exist positive constants c_1 , c_2 , and n_0 such that:

$$0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n \geq n_0$$

$f(n)$ is sandwiched between two constant multiples of $g(n)$ forever beyond n_0 . The function $g(n)$ is both a ceiling and a floor for $f(n)$ up to constant factors.

Big-Theta Notation(Θ)

$f(n) = \Theta(g(n))$ means function $f(n)$ is equal to $g(n)$ within a constant factor and $g(n)$ is asymptotically tight bound for $f(n)$.

We may write $f(n) = \Theta(g(n))$ OR $f(n) \in \Theta(g(n))$

Set of all functions that have same rate of growth as $g(n)$.

Problem(Big O Notation)

Find the Upper Bound for $f(n) = 2n^2 + 5n + 80$

we must find positive constants 'c' and ' n_0 '

such that: $f(n) \leq c * g(n)$ for all $n \geq n_0$

$f(n) = 2n^2 + 5n + 80$, the highest power of n is n^2 . Therefore, our target $g(n) = n^2$.

$$2n^2 + 5n + 80 \leq 2n^2 + 5n^2 + 80n^2$$

$$2n^2 + 5n + 80 \leq 87n^2$$

$$\text{for } n = 1: 2(1)^2 + 5(1) + 80 \leq 87(1)^2$$

$$87 \leq 87 \text{ is true, we can use: } n_0 = 1.$$

Since there exist constants $c = 87$ and $n_0 = 1$ such that

$$f(n) \leq c * n^2 \text{ for all } n \geq n_0,$$

we conclude: $f(n) = O(n^2)$

Problem(Big O Notation)

$$f(n) = 5n + 50$$

$$g(n) = n$$

is $g(n)$ upper bound of $f(n)$?

We must find positive constants 'c' and ' n_0 '

such that: $f(n) \leq c * g(n)$ for all $n \geq n_0$

$f(n) = 5n + 50$, the highest power of n is n .

$$5n + 50 \leq 5n + 50n$$

$$5n + 50 \leq 55n$$

$$\text{For } n = 1: \quad 5(1) + 50 \leq 55(1)$$

$55 \leq 55$ is true, we can use: $n_0 = 1$.

Since there exist constants $c = 55$ and $n_0 = 1$

such that $f(n) \leq c * n$ for all $n \geq n_0$,

we conclude: $f(n) = O(n)$ Yes, $g(n) = n$ is the upper bound of $f(n)$.

Problem(Big O Notation)

Prove that $2n^2 \in O(n^3)$

Proof: Assume that $f(n) = 2n^2$, and $g(n) = n^3$

Now we have to find the existence of c and n_0

such that:

$$f(n) \leq c * g(n)$$

$$2n^2 \leq c * n^3$$

Divide both sides by n^2 :

$$2 \leq c * n$$

If we take:

$$c = 1 \text{ and } n_0 = 2 \text{ (Since } 2 \leq 1 * 2 \text{ is true)}$$

OR

$$c = 2 \text{ and } n_0 = 1 \text{ (Since } 2 \leq 2 * 1 \text{ is true)}$$

Then:

$$2n^2 \leq c * n^3$$

Hence $f(n) \in O(g(n))$ for $c = 2$ and $n_0 = 1$.

Problems(Big O Notation)

1. Find the Upper Bound for $f(n) = 4n^2 + 15n + 100$
2. Find the Upper Bound for $f(n) = 3n^2 + 10n + 5$
3. If $f(n) = 8n + 120$ and $g(n) = n$, is $g(n)$ an upper bound of $f(n)$?
4. $f(n) = 12n + 80$, $g(n) = n$ Is $g(n)$ the upper bound of $f(n)$?
5. $f(n) = 3n^2 + 20$, $g(n) = n^3$ Is $g(n)$ the upper bound of $f(n)$?
6. Prove that: $5n^2 \in O(n^2)$
7. Prove that: $10n + 100 \in O(n^2)$
8. Prove that $4n^2 \in O(n^3)$

Problems(Big Ω Notation)

1. Find the Lower Bound for $f(n) = 4n^2 + 15n + 100$
2. Find the Lower Bound for $f(n) = 3n^2 + 10n + 5$
3. If $f(n) = 8n + 120$ and $g(n) = n$, is $g(n)$ a lower bound of $f(n)$?
4. $f(n) = 12n + 80$, $g(n) = n$ is $g(n)$ the lower bound of $f(n)$?
5. $f(n) = 3n^2 + 20$, $g(n) = n$ Is $g(n)$ the lower bound of $f(n)$? Find at least two different pairs of (c, n_0) that satisfy the inequality.
6. Prove that: $5n^2 \in \Omega(n^2)$
7. Prove that: $10n^2 + 100 \in \Omega(n)$
8. Prove that: $4n^3 \in \Omega(n^2)$

Problems(Big Θ Notation)

1. Find the Tight Bound for $f(n) = 4n^2 + 15n + 100$
2. Find the Tight Bound for $f(n) = 3n^2 + 10n + 5$
3. If $f(n) = 8n + 120$ and $g(n) = n$, is $g(n)$ a tight bound of $f(n)$?
4. $f(n) = 12n + 80$, $g(n) = n$ Is $g(n)$ the tight bound of $f(n)$?
5. $f(n) = 3n^2 + 20$, $g(n) = n^2$ Is $g(n)$ the tight bound of $f(n)$?
6. Prove that: $5n^2 \in \Theta(n^2)$
7. Prove that: $10n + 100 \in \Theta(n)$
8. Prove that: $4n^3 + 2n \in \Theta(n^3)$

Little o (Strict Upper Bound)

Big-O allows **f** and **g** to grow at the same rate.

little-o does not — it requires **f** to grow *strictly* slower than **g**.

Formal Definition:

$f(n) = o(g(n))$ if for **every** positive constant c , there exists n_0 such that:

$$f(n) < c \cdot g(n) \text{ for all } n \geq n_0$$

Little o (Strict Upper Bound)

Notice the critical difference from Big-O. In Big-O, we said "*there exists a c.*" In little-o, we say "*for every c.*" This is a fundamentally stronger requirement. We need $f(n)$ to be dominated by $g(n)$ no matter how small we make the constant c . $g(n)$ dominates $f(n)$ so completely that even $c = 0.00001$ is eventually enough to keep g above f .

The limit characterization:

$f(n) = o(g(n))$ if and only if $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$

The ratio of f to g goes to zero. f becomes negligible compared to g as n grows.

Little ω (Strict Lower Bound)

little- ω is to Big- Ω what little- o is to Big- O . It is the strict version of the lower bound.

Formal Definition:

$f(n) = \omega(g(n))$ if for **every** positive constant c , there exists n_0 such that:

$$f(n) > c \cdot g(n) \text{ for all } n \geq n_0$$

The limit characterization:

$$f(n) = \omega(g(n)) \text{ if and only if } \lim_{n \rightarrow \infty} f(n)/g(n) = \infty$$

The ratio of **f** to **g** blows up to infinity. **f** dominates **g** so completely that no constant multiple of **g** can keep up with **f**.

Limit Characterization

Limit characterization is often called the *Calculus way* to determine asymptotic notation. Instead of hunting for constants c and n_0 we look at the ratio of two functions as n goes to infinity.

It is much faster for complex functions because you can use **L'Hôpital's Rule**.

We define the limit as:

$$L = \lim[n \rightarrow \infty] f(n)/g(n)$$

Limit Characterization

$$L = \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$$

Depending on the value of L , we can determine the relationship:

If the Limit L is...	Then the relation is...	Intuition
0	$f(n) = o(g(n))$ and $O(g(n))$	f grows much slower than g .
A constant $c > 0$	$f(n) = \Theta(g(n))$	f and g grow at the same rate.
∞	$f(n) = \omega(g(n))$ and $\Omega(g(n))$	f grows much faster than g .

If L is a constant $c > 0$ then $f(n)$ is also $O(g(n))$ and $\Omega(g(n))$ because Big Theta implies both

Limit Characterization

L'Hôpital's Rule

When we get an indeterminate form like ∞/∞ or $0/0$ we will take the derivative of the top and bottom separately:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{f'(n)}{g'(n)}$$

Limit Characterization

Problem: Compare $f(n) = 3n^2 + 5n$ and $g(n) = n^2$.

$$L = \lim (n \rightarrow \infty) [(3n^2 + 5n) / n^2]$$

Divide terms by n^2 :

$$L = \lim (n \rightarrow \infty) [3 + 5/n]$$

$$L = 3 + 0 = 3$$

Result: $f(n) = \Theta(n^2)$ because the limit is a positive constant.

Limit Characterization

Problem: Compare $f(n) = 100n$ and $g(n) = n^2$.

$$L = \lim_{n \rightarrow \infty} [100n / n^2]$$

Simplify the fraction:

$$L = \lim_{n \rightarrow \infty} [100 / n]$$

$$L = 100 / \infty = 0$$

Result: $f(n) = o(n^2)$ because the limit is 0 (f grows slower than g).

Limit Characterization

Problem: Compare $f(n) = n^2$ and $g(n) = \ln(n)$.

$$L = \lim_{n \rightarrow \infty} [n^2 / \ln(n)] \rightarrow (\text{Form: } \infty/\infty)$$

Apply L'Hôpital's Rule (Derivatives):

Derivative of n^2 is $2n$

Derivative of $\ln(n)$ is $1/n$

$$L = \lim_{n \rightarrow \infty} [2n / (1/n)]$$

$$L = \lim_{n \rightarrow \infty} [2n^2]$$

$$L = 2(\infty)^2 = \infty$$

Result: $f(n) = \omega(\ln n)$ because the limit is ∞ (f grows faster than g).

Relations Over Asymptotic Notations

Reflexive Relation:

Let X be a non-empty *set* and R is a relation over X then R is said to be reflexive if $(a, a) \in R, \forall a \in X$,

Example 1: Let G be a graph. Let us define a relation R over G as if node x is connected to y then $(x, y) \in G$.

Reflexivity is satisfied over G if for every node there is a self loop.

Example 2: Let P be a set of all persons, and S be a relation over P such that if $(x, y) \in S$ then x has same birthday as y . Of course this relation is reflexive because $(x, x) \in S, \forall a \in P$,

Relations Over Asymptotic Notations

Reflexivity:

$f(n) = O(f(n))$ every function is Big-O of itself

Since, $0 \leq f(n) \leq cf(n)$ $\forall n \geq n_0 = 1$, if $c = 1$

$f(n) = \Omega(f(n))$ every function is Big- Ω of itself

Since, $0 \leq f(n) \leq cf(n)$ $\forall n \geq n_0 = 1$, if $c = 1$

$f(n) = \Theta(f(n))$ every function is Big- Θ of itself

Since, $0 \leq c_1 f(n) \leq f(n) \leq c_2 f(n)$ $\forall n \geq n_0 = 1$, if $c_1 = c_2 = 1$

Relations Over Asymptotic Notations

Little $\mathbf{o}$ and $\mathbf{\omega}$ are not Reflexivity Relations

Example

As we can not prove that $f(n) < f(n)$, for any n , and for all $c > 0$
Therefore

1. $f(n) \neq o(f(n))$
2. $f(n) \neq \omega(f(n))$

Hence small $\mathbf{o}$ and small $\mathbf{\omega}$ are not reflexive relations

Relations Over Asymptotic Notations

Symmetry

Let X be a non-empty set and R is a relation over X then R is said to be symmetric if $\forall a, b \in X, (a, b) \in R \implies (b, a) \in R$

Example 1:

Let P be a set of persons, and S be a relation over P such that if $(x, y) \in S$ then x has the same sign as y .

This relation is symmetric because $(x, y) \in S \implies (y, x) \in S$

Example 2:

Let P be a set of all persons, and B be a relation over P such that if $(x, y) \in B$ then x is brother of y .

This relation is not symmetric because $(\text{Anwer}, \text{Sadia}) \in B \implies (\text{Sadia}, \text{Brother}) \notin B$

Relations Over Asymptotic Notations

Symmetry over Θ Property:

Prove that $f(n) = \Theta(g(n)) \iff g(n) = \Theta(f(n))$

Proof:

By definition of Θ , if $f(n) = \Theta(g(n))$, there exist constants $c_1, c_2 > 0$:

$$0 \leq c_1 * g(n) \leq f(n) \leq c_2 * g(n) \text{ for all } n \geq n_0$$

From the right side

$$(f(n) \leq c_2 * g(n)):$$

Divide by c_2 :

$$(1/c_2) * f(n) \leq g(n)$$

Relations Over Asymptotic Notations

From the left side

$$(c_1 * g(n) \leq f(n))$$

Divide by c_1 :

$$g(n) \leq (1/c_1) * f(n)$$

Combining these results:

$$0 \leq (1/c_2) * f(n) \leq g(n) \leq (1/c_1) * f(n)$$

Let $c_3 = 1/c_2$ and $c_4 = 1/c_1$:

$$0 \leq c_3 * f(n) \leq g(n) \leq c_4 * f(n)$$

This matches the definition of Θ :

$$g(n) = \Theta(f(n)) \text{ Hence, } f(n) = \Theta(g(n)) \iff g(n) = \Theta(f(n))$$

Relations Over Asymptotic Notations

Transitivity:

Let X be a non-empty set and R is a *relation* over X then R is said to be transitive if $\forall a, b, c \in X, (a, b) \in R \wedge (b, c) \in R \implies (a, c) \in R$

Example 1:

Let P be a set of all persons, and B be a *relation* over P such that if $(x, y) \in B$ then x is brother of y .

This *relation* is transitive this is because $(x, y) \in B \wedge (y, z) \in B \implies (x, z) \in B$.

Example 2:

Let P be a set of all persons, and F be a *relation* over P such that if $(x, y) \in F$ then x is father of y .

Of course this *relation* is not a transitive because

$$\text{if } (x, y) \in F \wedge (y, z) \in F \implies (x, z) \notin F$$

Relations Over Asymptotic Notations

Transitivity:

If $f = O(g)$ and $g = O(h)$, then $f = O(h)$

If $f = \Omega(g)$ and $g = \Omega(h)$, then $f = \Omega(h)$

If $f = \Theta(g)$ and $g = \Theta(h)$, then $f = \Theta(h)$

Same holds for little-o and little- ω

Transitivity lets us chain complexity comparisons, which will be used constantly when reasoning about composed *algorithms*.

Relations Over Asymptotic Notations

Transpose symmetry:

$f(n) = O(g(n))$ if and only if $g(n) = \Omega(f(n))$

$f(n) = o(g(n))$ if and only if $g(n) = \omega(f(n))$

This one is deeply intuitive. If **f** is an upper bound on **g**, then **g** is a lower bound on **f**.

If **f** grows strictly slower than **g**, then **g** grows strictly faster than **f**. The direction flips when you flip the relationship.

Relations Over Asymptotic Notations

Transpose symmetry:

Prove that $f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$

Proof

Since $f(n) = O(g(n)) \Rightarrow \exists$ constants $c > 0$ and $n_0 \in \mathbb{N}$

such that $0 \leq f(n) \leq cg(n) \quad \forall n \geq n_0$

Dividing both sides by c :

$$0 \leq (1/c)f(n) \leq g(n) \quad \forall n \geq n_0$$

Putting $1/c = c'$:

$$0 \leq c'f(n) \leq g(n) \quad \forall n \geq n_0$$

Hence, $g(n) = \Omega(f(n))$

Relations Over Asymptotic Notations

Transpose symmetry:

Prove that $f(n) = o(g(n)) \Leftrightarrow g(n) = \omega(f(n))$

Proof

Since $f(n) = o(g(n)) \Rightarrow \exists$ constants $c > 0$ and $n_0 \in \mathbb{N}$

such that $0 \leq f(n) < cg(n) \quad \forall n \geq n_0$

Dividing both sides by c :

$$0 \leq (1/c)f(n) < g(n) \quad \forall n \geq n_0$$

Putting $1/c = c'$:

$$0 \leq c'f(n) < g(n) \quad \forall n \geq n_0$$

Hence, $g(n) = \omega(f(n))$

Big-O is not a tight bound. Neither is Big- Ω .

When an *algorithm* runs in $\Theta(n \log n)$ time, all of the following are technically true:

It is	$O(n \log n)$	correct and tight
It is	$O(n^2)$	correct but loose
It is	$O(n^{100})$	correct but absurdly loose
It is	$\Omega(n)$	correct but loose
It is	$\Omega(\log n)$	correct but very loose

Big-O is not a tight bound. Neither is Big- Ω .

None of those *loose bounds* are wrong. They are just not useful. The reason we care about *tight bounds* Θ is that they give the most information.

A *loose* Big-O tells something. A *tight* Θ tells everything about growth behavior.

When someone says an algorithm is $O(f(n))$, they usually mean it as a *tight bound* even if they are technically only stating an *upper bound*.

The Standard Complexity Classes

$O(1)$ — Constant time: The *algorithm* does the same amount of work regardless of input size. Array index access, hash table lookup (average case).

$O(\log n)$ — Logarithmic time: The work grows *logarithmically*. Binary Search is the canonical example.

$O(n)$ — Linear time: Work grows *proportionally* with input. Linear Search, finding the maximum in an unsorted array.

$O(n \log n)$ — Linearithmic time: The *sweet* spot for many important problems. Merge Sort, Heap Sort, and many other optimal algorithms live here.

The Standard Complexity Classes

$O(n^2)$ — Quadratic time: Work grows as the *square* of input size. Bubble Sort, Insertion Sort (worst case), simple graph algorithms.

$O(n^3)$ — Cubic time: Naive *matrix multiplication*. Only practical for small to medium inputs.

$O(2^n)$ — Exponential time: Grows *catastrophically*. Exact solutions to many NP-hard problems.

$O(n!)$ — Factorial time: *Brute-force* combinatorial search.